

An Overview of Open Source Databases

Every organisation keeps data. Over the last 50 years, electronic record-keeping, powered by Codd's relational model, has become the foundation of the modern data driven business. Making decisions or defining strategies requires analysis of this data ('at rest'), to discover patterns that can be leveraged using modern statistical modelling approaches. Online analytical processing (OLAP) and data warehousing techniques are representative of this approach.

Due to the increased competition, organisations need to process their data ('in motion') at very short time intervals. With the current focus on volume, variety and velocity of data, newer architectures have come up that seek to emphasise Brewer's CAP theorem while relaxing the stringent requirements of a two-phase commit model. As exemplified by the NoSQL (Not only SQL) movement, it is common to see discussions revolve around Elasticsearch, Redis, Riak, MongoDB, among others, at start-up and tech community events.

The other driver is the machine learning, analytics and data science community, and companies that work with sophisticated models in tandem with business. Since the data is very large, most of the compute and storage infrastructure is powered on clusters humming along with Hadoop, Spark, etc, in the cloud.

Thus, there is a spectrum of architectural choices that span from disk-oriented, row-centric, relational models and column-oriented data warehouses for OLAP, to distributed key-value or document stores and new SQL models with high concurrency requirements.

The other set of choices an organisation needs to make relates to the tolerance to data loss, storage failure, inconsistent state issues, lost transactions and poor performance metrics, depending on where the product or the service is in terms of the number of customers. Thus, as the traffic builds up, new challenges and bottlenecks are identified.

Yet another set of choices emanate from the technical debt that your team has accumulated. Thus, a hack using MongoDB by a start-up (either due to lack of design skills or paucity of time) is different from a team that decides to use static typing, interfaces and data structures explicit in its programming model to exploit the powerful capabilities of Redis key-value store. There could be another team with high transactional integrity requirements, which uses a JSON-friendly distributed data store.

Due to the large diversity of data-centric applications

and products that are being built from the ground up, decisions and choices have to be made with respect to the database that should be used. This article lists some of those choices while keeping the spotlight on data (data at rest vs data in motion).

Relational model

The notion of data is one of 'data at rest' as in relational databases like MySQL, PostgreSQL etc.

The various components, transaction logs are highlighted in the Figure 1. There is the familiar SQL interface for the database OLTP and OLAP capabilities are consumed through a combination of drivers, modules, SQL extensions, application programming interfaces and language bindings.

